

Chapter 2

第二章初值问题

参考视频:

初值问题:

https://www.bilibili.com/video/BV1jZ4y1Z72B/?spm_id_from=333.999.0.0

数据插值:

https://www.bilibili.com/video/BV1Bs4y187iu/?spm_id_from=333.999.0.0&vd_source=92fe40ef7838946864ed830f972e2f3a

2.1 初值问题的数值解法

在求解微分方程的初值问题时，我们需要得到物理量随时间的变化。在数值方法中，我们只需要建立物理量和特定时刻的对应即可，比如生成时间序列，根据微分方程得到物理量对应的序列。通常先将时间离散化，在有限的时间间隔内，将微分方程中的各阶导数用差分形式表示（Euler 方法）：

$$\frac{dx}{dt} = f(x, t) \rightarrow x_{i+1} = x_i + f(x_i, t_i)\Delta t \quad (2.1)$$

根据给定的初始条件，类似数列的计算方法，用 for 循环处理即可。

2.1.1 初值问题的微分方程和差分形式

稳定的核素对核子总数有一定限度，而且中子数和质子数应保持一定的比例。不稳定的核素会自发地放射出 α 射线，而转变成原子序数较小的新核；或是因核素的核子总数和原子序数的比率不适当，其核内的中子与质子会自发地相互转变，并同时放出一个 β^- （或 β^+ ）粒子。可以用微分方程描述未衰变数目 (N) 随时间 (t) 的变化：

$$\frac{dN}{dt} = -\frac{N}{\tau} \quad (2.2)$$

其中 τ 是常数，和核素的种类有关。

对于理想单摆，在偏角小的条件下，可以推导得到偏转角度 (θ) 满足的微分方程：

$$\frac{d^2\theta}{dt^2} = -\frac{g}{L}\theta \rightarrow \begin{cases} \frac{d\theta}{dt} = \omega \\ \frac{d\omega}{dt} = -\frac{g}{L}\theta \end{cases} \quad (2.3)$$

其中 g 和 L 分别是重力加速度和摆长，引入了角速度 ω ，将二阶微分方程变成了两个一阶微分方程。

在忽略空气阻力的情况下，我们以一定速度 v_0 将物体抛出，可以看到物体位置随时间变化。在物体到达地面之前，物体的速度、位移都是可以通过理论计算确定的。由于物体只受重力作用，我们可以根据速度方向将运动分为竖直上抛运动和斜抛运动（平抛运动是斜抛运动的特例），分别对应的是一维和二维问题。根据运动独立性原理，可以把斜抛运动看成是作水平方向的匀速直线运动和竖直上抛运动的合运动来处理。当速度方向与地面夹角为 θ 时，运动微分方程从竖直上抛过渡到斜抛形式：

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -g \end{cases} \rightarrow \begin{cases} \frac{dv_x}{dt} = 0, \frac{dx}{dt} = v_x \\ \frac{dv_y}{dt} = -g, \frac{dy}{dt} = v_y \end{cases} \quad (2.4)$$

当体系存在约束情况，约束力通常无法直接表示，如图（d）所示，重力分量和支持

力提供向心力。可以根据能量关系，分析圆周运动和抛体运动临界条件。

$$\frac{1}{2}mv_0^2 = \frac{1}{2}mv^2 + mgR(1 - \cos \theta) \quad (2.5)$$

其中 $\frac{1}{2}mv_0^2$ 是初始动能， $\frac{1}{2}mv^2$ 是当前动能， $mgR(1 - \cos \theta)$ 是势能的变化量。

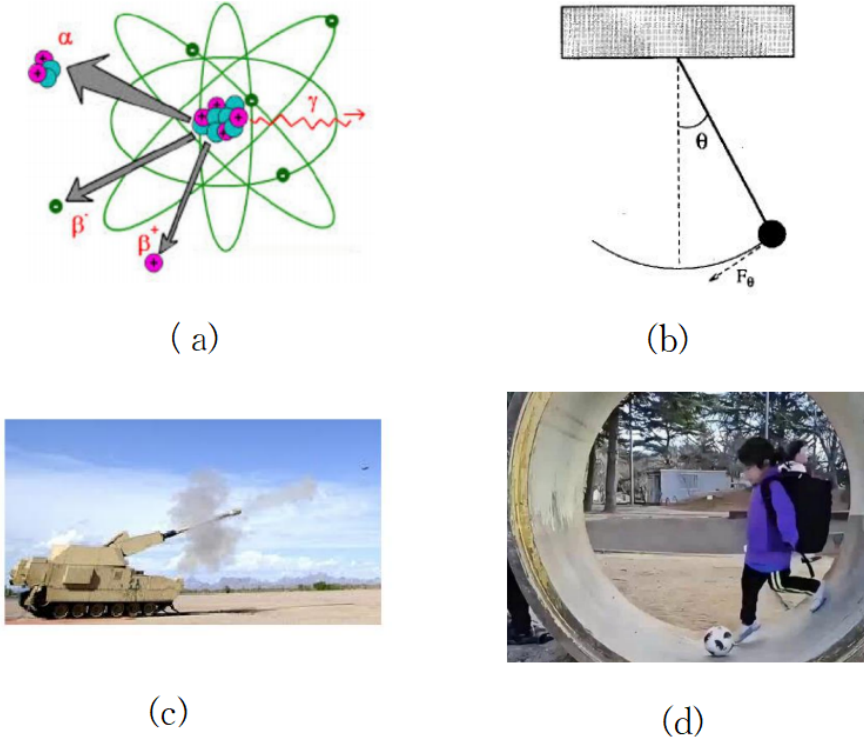


图 2.1: 初值问题举例：(a) 衰变问题；(b) 单摆运动；(c) 斜抛运动；(d) 圆周运动。

在这些微分方程中，关键是要处理一阶微分方程问题。以衰变问题为例，考虑粒子数目随时间变化的泰勒展开式：

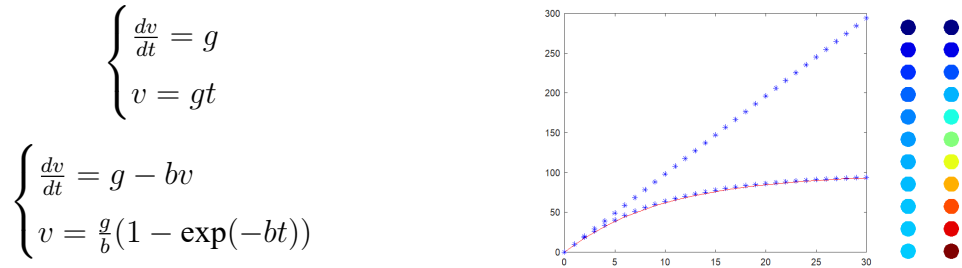
$$N(t + \Delta t) = N(t) + \frac{dN}{dt}\Delta t + \frac{1}{2}\frac{d^2N}{dt^2}\Delta t^2 + \dots \quad (2.6)$$

舍弃高阶项，结合微分方程，得到在 $t + \Delta t$ 时刻的粒子数目和导数的关系：

$$N(t + \Delta t) \approx N(t) - \frac{N(t)}{\tau}\Delta t \quad (2.7)$$

值得注意的是， Δt 是影响计算结果的主要因素，需要仔细测试结果随划分的收敛性。另外，我们可以通过物理量的取值范围（粒子数非负）来判断结果的合理性。

在日常生活中，我们看到树枝比树叶下落的快，容易误解是重的物体下落得比较轻的物体快。在伽利略著名的斜塔实验中，人们看到了两个铁球同时着地，也证明了地球表面的重力加速度是常数。而树枝和树叶的下落速度不同，主要来自于空气阻力的影响。空气阻力通常和速度有关，这里以一次和二次函数为例。在自由落体运动中，我们选用的是阻力大小和速度关系是一次函数，此时速度随时间变化有简单的理论解。在抛射体运动中，我们选用的是阻力大小和速度关系是二次函数，重点看阻力沿不同方向的分解处理。对于自由落体运动，下图给出考虑空气阻力前后的物体运动的微分方程和对应的解，曲线显示了速度随时间变化，不同颜色的球代表物体下落速度，越红对应速度越大。



当物体在大气中移动时，必须将前方的空气推开。在时间 dt 内被移动的空气质量为： $m_{\text{air}} \sim \rho A v dt$ 其中 ρ 是空气密度， A 是物体的横截面积。这些空气被赋予了大约 v 的速度，因此其动能为： $E_{\text{air}} \sim \frac{1}{2} m_{\text{air}} v^2$ 这同时也是阻力（由于空气阻力作用在物体上的力）在时间 dt 内所做的功，因此： $F_{\text{drag}} v dt = E_{\text{air}}$ 将这些综合起来，我们得到：

$$F_{\text{drag}} \approx -\frac{1}{2} C \rho A v^2 \quad (2.8)$$

其中 C 被称为拖曳系数，一般情况为 1。需要注意的是，这里计算只是近似的，定性反映 F_{drag} 对 A 和 v 的函数依赖关系。

对于斜抛运动，假定摩擦力的大小正比于速度的平方 (系数 b 对应的是单位质量)，注意到摩擦力方向和速度在同一直线上，随时间变化。我们将摩擦力沿水平和竖直方向，计算速度的大小 $v = \sqrt{v_x^2 + v_y^2}$ ，并根据 v_x/v 、 v_y/v 分别计算速度和水平方向夹角的余弦、正弦即可。对应的运动方程如下：

$$\begin{cases} \frac{dv_x}{dt} = -v v_x, \frac{dx}{dt} = v_x \\ \frac{dv_y}{dt} = -g - b v v_y, \frac{dy}{dt} = v_y \end{cases} \quad (2.9)$$

当运动物体有明显的自转时，比如乒乓球、网球、足球等，运动轨迹将和之前的抛物线轨迹有明显的偏离，此时是因为马格努斯力的影响。如图 2.4(a) 所示，将物体近似看成圆，圆周上不同位置的线速度不同，最终出现大小和自转角速度、平动速度、半径乘积成正比的力。严格的计算公式是 2.14 给出的矢量方式，并由此可知力的方向。

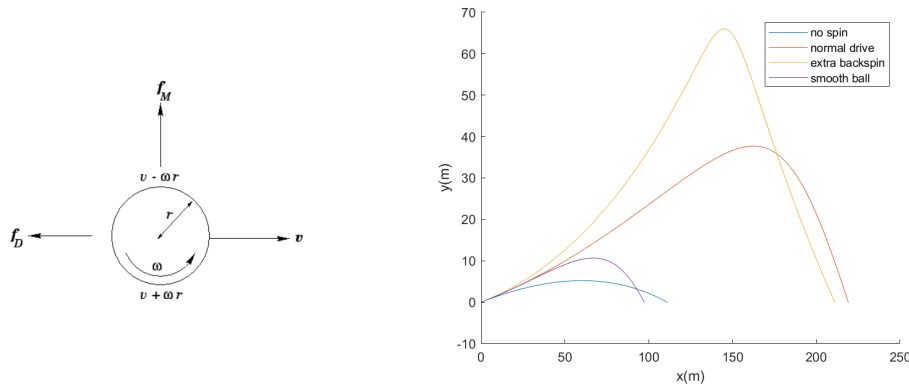


图 2.2: 马格努斯力: (a) 示意图; (b) 高尔夫球运动轨迹。

$$\vec{F}_M = S_0 \vec{\omega} \times \vec{v} \rightarrow F_M \propto (v + r\omega)^2 - (v - r\omega)^2 \propto vr\omega \quad (2.10)$$

以高尔夫球的运动为例，我们使用一个坐标系，其中 y 是球离地面的高度， x 是与球的初始速度在同一平面内的水平方向，而 z 是与速度垂直的水平方向。高尔夫球的运动方程如下：

$$\frac{dv_x}{dt} = -\frac{F_{\text{drag},x}}{m} - \frac{S_0 \omega v_y}{m}, \quad \frac{dv_y}{dt} = -\frac{F_{\text{drag},y}}{m} + \frac{S_0 \omega v_x}{m} - g \quad (2.11)$$

通常还有 dx/dt 和 dy/dt 的方程。这里我们假设球被击中时带有后旋 (backspin)，角速度为 ω ，因此旋转轴沿 z 方向，从而与 x 和 y 垂直。对真实高尔夫球的测量估算，阻力系数 C 在低速时 $C \approx \frac{1}{2}$ ，但随着速度的增加，这个系数急剧下降，在高速时测得 $C \approx 7.0/v$ (其中 v 的单位为 m/s)。在模型中包含这两种极限情况的行为，假设：

$$F_{\text{drag}} = -C\rho A v^2 \quad (2.12)$$

其中，当速度 $v \leq 14 \text{ m/s}$ 时， $C = \frac{1}{2}$ ；当速度更高时， $C = 7.0/v$ 。高尔夫球的参数： $\rho = 1.1691 \times 10^3 \text{ kg/m}^3$, $A = \pi(0.0213)^2 \text{ m}^2$, $m = 0.0456 \text{ kg}$ 。这里的 $S_0 \omega$ 是整体参数，取零对应的是 no spin，一般取 0.25 对应的是 normal drive，取 0.25×1.5 对应的是 extra backspin，而 'smooth ball' 对应的是 $C = 0$ 。可见，要打出高远球，要求高尔夫球具有高速的自转，而且球的表面有很多“凹陷”，主要是让球在平动速度较大时，系数 C 有明显的下降。

2.1.2 数据插值

在数值方法中，我们对时间、空间都做了有限宽度划分，在求解问题时，我们得到物理量随特定时刻、位置的分布（采样是无法做到连续的）。如果我们有问题的解析解，比如物理量随时间、空间变化的函数，直接代入即可。多数时候，我们采用数值方法是因为该问题不存在解析解或者说不存在简单的函数形式，比如 $dt = 0.5$ ，如何计算 $t = 2.1$ 的数值？当斜抛物体到达地面时，我们知道 $y = 0$ ，但是程序结果是 $y_{n-1} > 0, y_n < 0$ ，如何根据 x_{n-1}, x_n 确定落地点的位置？通常，需要借助数据插值来解决。假设落地点的坐标为 $(x_L, 0)$ ，根据三点共线斜率相同，可以得到坐标 x_L 和 (x_{n-1}, y_{n-1}) 、 (x_n, y_n) 的关联：

$$\frac{y_{n-1} - y_n}{x_{n-1} - x_n} = \frac{y_n}{x_n - x_L} \rightarrow x_L = \frac{x_{n-1} + r x_n}{r + 1}, r = -\frac{y_{n-1}}{y_n} \quad (2.13)$$

在数据插值中，最简单的就是线性插值。已知两点坐标为 (x_1, y_1) 和 (x_2, y_2) ，可以通过直线斜率和截距得到直线方程：

$$y = kx + b, k = \frac{y_2 - y_1}{x_2 - x_1}, b = \frac{x_2 y_1 - x_1 y_2}{x_2 - x_1}, y = y_1 \frac{x - x_2}{x_1 - x_2} + y_2 \frac{x - x_1}{x_2 - x_1} \quad (2.14)$$

如上所示，该函数由两部分组成，当 $x = x_1$ 时，第一部分返回 y_1 ，第二部分返回 0；当 $x = x_2$ 时，第二部分返回 y_1 ，第一部分返回 0。对应由三点 [坐标为 $(x_1, y_1), (x_2, y_2), (x_3, y_3)$] 组成的二次函数，我们可以直接写出：

$$y = y_1 \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} + y_2 \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} + y_3 \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} \quad (2.15)$$

这样的写法可以避免求解多项式的系数，直接根据坐标构造符号条件的多项式，类似思路推广到多点情况，对应的是拉格朗日插值法。

值得注意的是，实际使用中我们更常用的是样条插值，选择分段函数，保证连接的光滑性。在平时的使用中，我们掌握插值函数的语法即可，这里主要介绍 Octave 里面的 1 维、2 维插值，分别对应 `spline`、`interp1` 和 `interp2` 函数。

```
1 a=[26.8892 -3.2733
2    36.8850 -3.8328
3    49.0939 -3.5725
4    63.7373 -3.0755];
5 plot(a(:,1),a(:,2),'o-')
6 hold on
7 x=linspace(min(a(:,1)),max(a(:,1)),200);
8 y=spline(a(:,1),a(:,2),x);
9 plot(x,y,'r')
10 [q,qq]=min(y);x(qq)
```

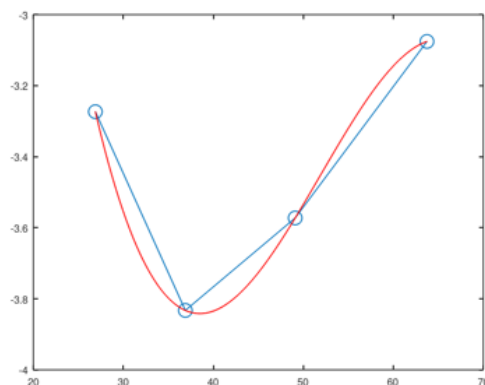


图 2.3: 一维插值函数

在上面程序中， a 的两列分别是已知数据的自变量、因变量，如果我们直接用直线连接就是蓝线。 $x=\text{linspace}(\min(a(:,1)),\max(a(:,1)),200)$; 是把自变量区间生成 200 个点， $y=\text{interp1}(a(:,1),a(:,2),x,\text{'spline'})$; 是调用插值函数 interp1 ，注意语法，里面有 4 个参数，分别对应已知数据的自变量、因变量、我们关心的自变量、插值方法。这里的 'spline' 是样条插值方法，对应分段多项式，并有一定光滑的连接要求，具体思路可以查阅相应书籍。

其中第 9 行可以换成 $y=\text{spline}(a(:,1),a(:,2),x)$; 在 Octave 中 spline 是独立的插值函数，得到的结果是相同的。 y 是插值后的因变量（红线），通过 min 函数查找最小值， $[q,qq]=\text{min}(y)$ ，这里 q 返回最小值， qq 返回最小值所在位置对应的整数，所以 $x(qq)$ 就是让 y 最小的自变量。

Octave 的二维插值函数是 interp2 ，但要注意自变量是矩阵，因为此时是矩阵之间的映射。 x 和 y 是 1 维向量，通过 for 循环计算得到 $V(i,j) = \exp(-((i-6)^2 + (j-6)^2)/10)$; 此时矩阵 V 是 11×11 的大小。因为网格稀疏，用 image 得到图片有明显的方格。先对 x 、 y 加密（增加划分点），然后用 meshgrid 得到对应加密后的网格，最后通过第 12 行的指令 interp2 实现二维插值得到 z_i ，对应矩阵是 501×501 的大小，这里 'cubic' 也是可以选择不同方法， help interp2 查看说明即可。

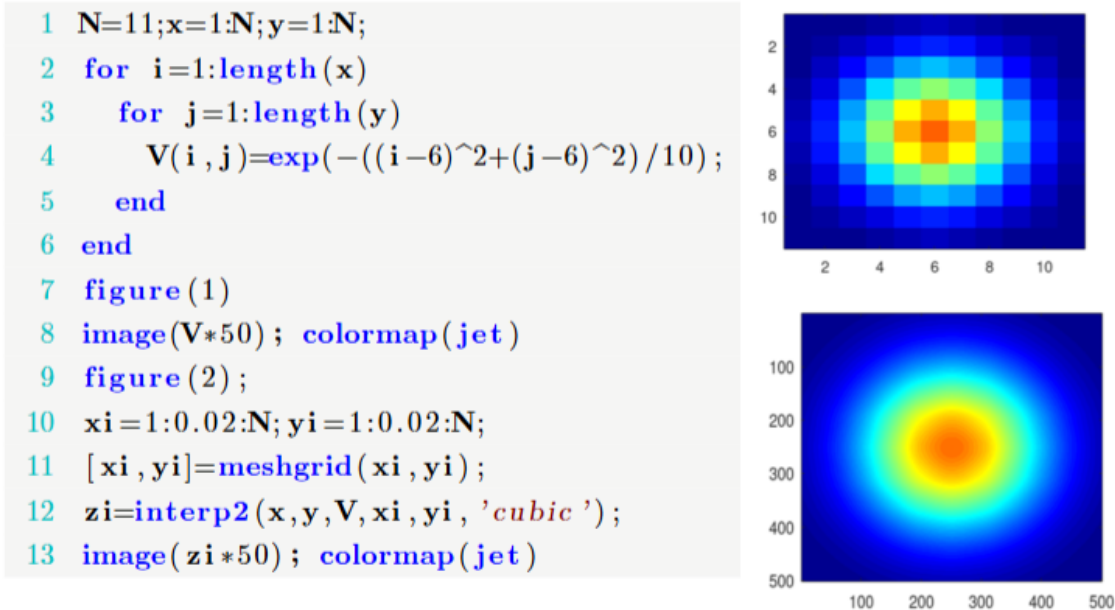


图 2.4: 二维插值函数

对于高维（3 个分量以上）情况，可以采用径向基函数插值（Radial Basis Function Interpolation, RBF 插值），结合高斯函数、多项式函数等来逼近给定数据点，基本形式为：

$$f(\mathbf{x}) = \sum_{i=1}^N w_i \varphi(\|\mathbf{x} - \mathbf{x}_i\|), \varphi(r) = e^{-\frac{r^2}{2\sigma^2}} \quad (2.16)$$

$\varphi(r)$ 是径向基函数（这里用的是高斯函数）， $\|\mathbf{x} - \mathbf{x}_i\|$ 向量之间的欧几里得距离。 $\mathbf{x}_i$ 是已知数据点， w_i 是权重系数， N 是数据点的数量。

2.2 抛射体问题的程序说明和参数相图

以重力作用下的物体运动为例，系统的加速度来自重力和空气阻力等，我们可以写出物体的运动方程，对应的是二阶微分方程。在一维情况下，可以得到：

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = f(x, v, t) \end{cases} \rightarrow \begin{cases} v(i+1) = v(i) + f(i)\Delta t \\ x(i+1) = x(i) + v_x(i+1)\Delta t \end{cases} \quad (2.17)$$

引入速度和加速度的关系，把二阶微分方程变成两个一阶微分方程，分别用差分方法可以完成递推求解。类似的方法可以推广到高阶微分方程，对于初值问题都是可以从初值开始，通过微分方程的差分形式进行递推，实现常见的动力学过程模拟，得到不同物理量随时间的变化，根据不同初始条件确定抛射体的落地点，讨论最大发射角和摩擦系数的关联。

2.2.1 抛射体轨迹和落地点的确定

在忽略空气阻力的情况下，我们以一定速度 v_0 将物体抛出，可以看到物体位置随时间变化。在物体到达地面之前，物体的速度、位移都是可以通过理论计算确定的。由于物体只受重力作用，我们可以根据速度方向将运动分为竖直上抛运动和斜抛运动（平抛运动是斜抛运动的特例），分别对应的是一维和二维问题。

当速度方向与地面垂直时，竖直上抛的运动微分方程和对应的解如下：

$$\begin{cases} \frac{dx}{dt} = v \\ \frac{dv}{dt} = -g \end{cases} \rightarrow \begin{cases} v(t) = v_0 - gt \\ x(t) = v_0 t - \frac{1}{2}gt^2 \end{cases} \quad (2.18)$$

当速度方向与地面夹角为 θ 时，竖直上抛的运动微分方程和对应的解如下：

$$\begin{cases} \frac{dv_x}{dt} = 0 \\ \frac{dx}{dt} = v_x \\ \frac{dv_y}{dt} = -g \\ \frac{dy}{dt} = v_y \end{cases} \rightarrow \begin{cases} v_x(t) = v_0 \cos \theta \\ v_y(t) = v_0 \sin \theta - gt \\ x(t) = v_0 t \cos \theta \\ y(t) = v_0 t \sin \theta - \frac{1}{2}gt^2 \\ y = x \tan \theta - \frac{gx^2}{2(v_0 \cos \theta)^2} \end{cases} \quad (2.19)$$

对于复杂的微分方程，我们通常难以得到解析表达式，需要通过数值方法解法建立物理量和时间的对应。假定将时间离散为 n 个分量的数组 $t = [t_1, t_2, \dots, t_n]$ ，目标是计算速度和位置 $v = [v_1, v_2, \dots, v_n]$ ， $x = [x_1, x_2, \dots, x_n]$ 。

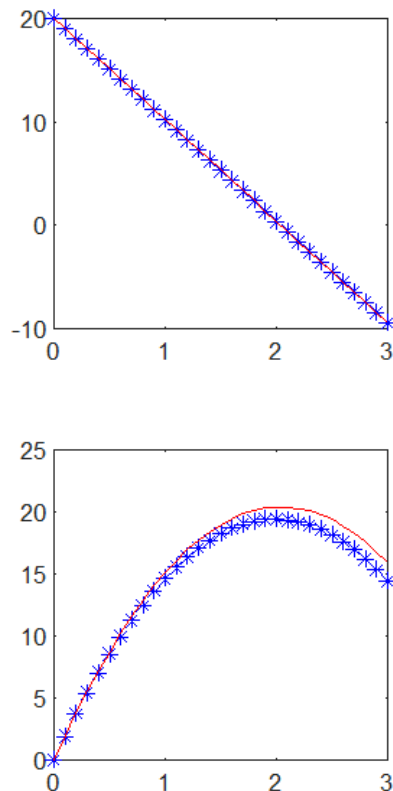
在 `for` 循环中，程序是设定了迭代步数，总的时间长度是最大步数乘以时间间隔。当 Δt 取不同数值时，如果要模拟同一时段的运动，我们需要调整不同的迭代步数，下面程序用了 `round` 取整函数（四舍五入），比如每次都是模拟到 $t = 3$ ，这里就让 j 循环到最接近 $(3/\text{dt})$ 的数值即可。这里的 `subplot` 是将两个子图放在一起显示，其中 $(2, 1, 1)$ 表

示子图变成 2 行 1 列摆放, 当前是第一个图, 我们画的是速度随时间变化, 计算结果和理论解完全符合。第二张图画的是位置随时间的变化, 基本是符合的, 但是误差随时间有一定的增大, 主要是目前的差分方法精度比较低。

```

1 clear
2 g=9.8;dt=0.1;
3 t(1)=0;v(1)=20;x(1)=0;
4 for j=1:round(3/dt)
5     v(j+1)=v(j)-g*dt;
6     x(j+1)=x(j)+v(j+1)*dt;
7     t(j+1)=t(j)+dt;
8 end
9 subplot(2,1,1)
10 plot(t,v,'*b')
11 hold on
12 plot(t,v(1)-g*t,'r')
13 subplot(2,1,2)
14 plot(t,x,'*b')
15 hold on
16 plot(t,v(1)*t-0.5*g*t.^2,'r')

```



对于斜抛运动（忽略空气阻力时），物体作匀变速曲线运动，轨迹是抛物线，通常把斜抛运动看成是作水平方向的匀速直线运动和竖直上抛运动的合运动来处理。根据微分方程，可以得到相应的差分形式：

$$\begin{cases} \frac{dv_x}{dt} = f_x \\ \frac{dx}{dt} = v_x \\ \frac{dv_y}{dt} = f_y \\ \frac{dy}{dt} = v_y \end{cases} \rightarrow \begin{cases} v_x(i+1) = v_x(i) + f_x(i)\Delta t \\ x(i+1) = x(i) + v_x(i+1)\Delta t \\ v_y(i+1) = v_y(i) + f_y(i)\Delta t \\ y(i+1) = y(i) + v_y(i+1)\Delta t \end{cases} \quad (2.20)$$

下面斜抛运动模拟的参考程序：第 2 行是初始条件，速度大小、与水平方向的夹角，第 3、4 行是水平、竖直方向的初始位置、速度大小，第 5 行是时间间隔，重力常数等。当物体从地面抛出，经过一定时间后将回到地面，因此水平方向 $y < 0$ 意味着物体已经着地，这里用 while 循环来终止程序，第 7 至 10 行是运动微分方程对应的差分方法。

```

1 clear
2 theta=pi/3;v=5;
3 vx(1)=v*cos(theta);x(1)=0;
4 vy(1)=v*sin(theta);y(1)=0;

```

```

5 dt=0.01;g=9.8;t(1)=0;i=1;
6 while y(i)>=0 %物体没有到达地面之前的循环
7 vx(i+1)=vx(i);
8 vy(i+1)=vy(i)-(g)*dt;
9 x(i+1)=x(i)+vx(i+1)*dt;
10 y(i+1)=y(i)+vy(i+1)*dt;
11 t(i+1)=t(i)+dt;i=i+1;
12 end

```

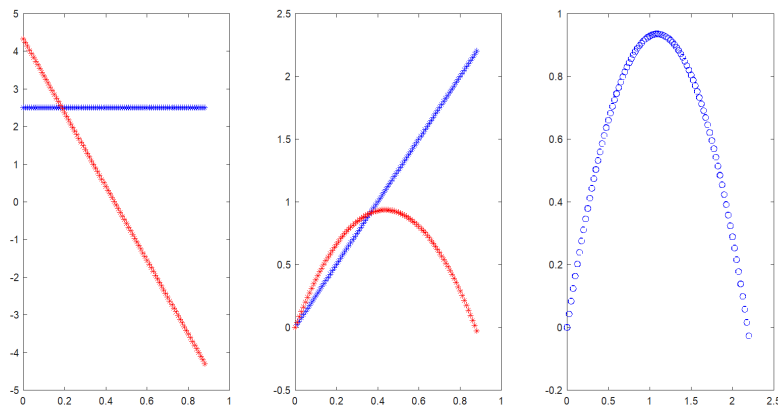


图 2.5: 斜抛运动模拟

当斜抛物体到达地面时, 我们知道 $y = 0$, 但是程序结果是 $y_{n-1} > 0, y_n < 0$, 落地地点的坐标为 $(x_L, 0)$, 根据公式 2.8 可以计算得到。以下程序是根据线性插值方法计算落地点, 输入发射角, 初始速度, 时间间隔, 返回射程的函数。在下面程序中, 当输入参数后, 第 2、3 行分别计算初始条件, 第 4 到第 11 行是模拟斜抛运动, 并在竖直方向位移小于 0 时结束程序, 第 12、13 行是使用线性插值方法计算落地点距离。如果使用库函数, 将插值部分修改为: `xl=interp1(y(end-1:end),x(end-1:end),0);` 或者 `xl=interp1(y(end-1:end),x(end-1:end),0,'spline');` 即可, 分别对应线性插值和 spline 函数的样条插值。

```

1 function xl=disfun(theta,v,dt)
2 g=9.8;t(1)=0;vx(1)=v*cos(theta);x(1)=0;
3 vy(1)=v*sin(theta);y(1)=0;i=1;
4 while y(i)>=0 %前面的while循环和上面求轨迹一致
5 vx(i+1)=vx(i);
6 vy(i+1)=vy(i)-g*dt;
7 x(i+1)=x(i)+vx(i+1)*dt;
8 y(i+1)=y(i)+vy(i+1)*dt;
9 t(i+1)=t(i)+dt;
10 i=i+1;
11 end
12 r=-y(end-1)/y(end); %是根据线性插值求落地点
13 xl=(x(end-1)+r*x(end))/(r+1);

```

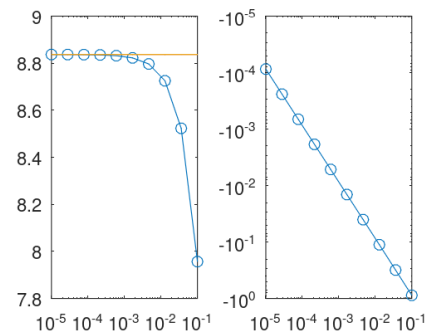
2.2.2 最佳发射角的确定

当初始条件确定时，我们完成了抛射体运动轨迹的模拟并确定了落地点位置。需要注意的是，时间的有限分割对结果带来误差。以落地点的计算为例，我们通过下面程序试落地点距离随 dt 的变化，逐渐逼近理论值 ($v_0^2 \sin(2\theta)/g$)，单位是米。当 dt 从 10^{-1} 到 10^{-5} 秒变化时，落地点距离逐渐接近理论解（红色线）。从理论值的偏差看，当 dt 达到 10^{-4} 秒时，和理论值偏差在 10^{-3} 米附近。

```

1 clear
2 dts=logspace(-1,-5,10);
3 for i=1:length(dts)
4     x1(i)=disfun(pi/6,10,dts(i));
5 end
6 subplot(1,2,1)
7 semilogx(dts,x1,'o-')
8 hold on
9 y=100*sin(pi/3)/9.8;
10 plot([dts(1) dts(end)],[y y])
11 semilogx(dts,y)
12 subplot(1,2,2)
13 loglog(dts,x1-y,'o-')

```

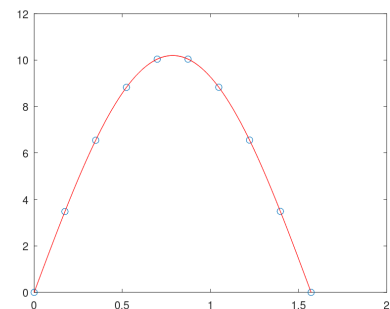


当完成了对 dt 的收敛性测试，我们可以进一步探讨在给定速度大小时，落地点最远对应的最佳发射角。考虑在 $[0, \pi/2]$ 之间将发射角进行离散化，对于给定的角度，初始速度大小、时间分割分别取 10 和 0.001，通过 for 循环计算对应的落地点距离，保存在 $x1$ 向量中，对应的是程序第 3 至 5 行。第 7 行是使用库函数 `max` 求 $x1$ 的最大值和最大值所在的向量位置，分别保存在变量 q 和 qq 中。根据理论值，我们知道最佳发射角是 $\pi/4$ ，但此时会有一定偏差。程序的第 10 行是数据插值，将角度变成 1000 个分割，可以得到更精确的结果，用插值避免反复调用 `disfun` 函数通过计算轨迹来确定落地点位置。这里的 $ts(qq)$ 和 $tt(qq)$ 将输出落地点最远对应的角度，而 $tt(qq)$ 将更接近理论的最佳发射角 $\pi/4$ 。可见，在参数扫描时，合理的数据插值可以提高精度和减少计算量。

```

1 clear
2 ts=linspace(0,pi/2,10);
3 for i=1:length(ts)
4     x1(i)=disfun(ts(i),10,0.001);
5 end
6 plot(ts,x1,'o')
7 [q,qq]=max(x1);
8 ts(qq)
9 tt=linspace(0,pi/2,1000);
10 xx=spline(ts,x1,tt);
11 [q,qq]=max(xx);
12 tt(qq)
13 hold on
14 plot(tt,xx,'r')

```



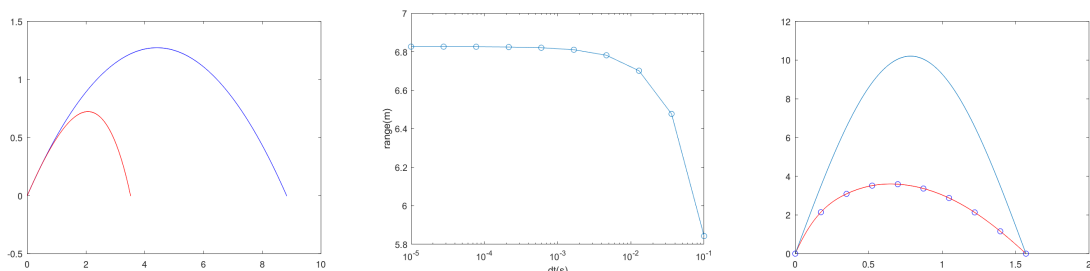
对于斜抛运动，假定摩擦力的大小正比于速度的平方 (系数 b 对应的是单位质量)，注意到摩擦力方向和速度在同一直线上，随时间变化。我们将摩擦力沿水平和竖直方向，计算速度的大小 $v = \sqrt{v_x^2 + v_y^2}$ ，并根据 v_x/v 、 v_y/v 分别计算速度和水平方向夹角的余弦、正弦即可。

$$\begin{cases} \frac{dv_x}{dt} = -vv_x \\ \frac{dx}{dt} = v_x \\ \frac{dv_y}{dt} = -g - bv v_y \\ \frac{dy}{dt} = v_y \end{cases} \rightarrow \begin{cases} v_x(i+1) = v_x(i) - v(i)v_x(i)\Delta t \\ x(i+1) = x(i) + v_x(i+1)\Delta t \\ v_y(i+1) = v_y(i) - (g + bv(i)v_y(i))\Delta t \\ y(i+1) = y(i) + v_y(i+1)\Delta t \end{cases} \quad (2.21)$$

下面给出的是参考程序。和忽略空气阻力的情况相比，这里系数 b 是需要提前确定，在 `while` 循环中，第 7 行是计算当前时刻的速度大小，第 8、9 行是根据上面微分方程和伪码修改的程序部分，其他部分和之前程序一样。

```
1 function xl=disfun2(theta,v,dt,b)
2 g=9.8;t(1)=0;
3 vx(1)=v*cos(theta);x(1)=0;
4 vy(1)=v*sin(theta);y(1)=0;
5 i=1;
6 while y(i)>=0
7 v(i)=sqrt(vx(i)^2+vy(i)^2);
8 vx(i+1)=vx(i)-v(i)*vx(i)*b*dt;
9 vy(i+1)=vy(i)-(g+b*v(i)*vy(i))*dt;
10 x(i+1)=x(i)+vx(i+1)*dt;
11 y(i+1)=y(i)+vy(i+1)*dt;
12 t(i+1)=t(i)+dt;
13 i=i+1;
14 end
15 r=-y(end-1)/y(end);
16 xl=(x(end-1)+r*x(end))/(r+1);
```

下图给出了初始条件相同时，考虑空气阻力对结果的影响，落地点距离明显变小 (左图)。类似忽略空气阻力的情况，我们对于确定的初始速度大小 $v = 10$ ，测试了不同 dt 对应的落地点收敛情况，发现在 $dt=0.001$ 秒时，精度达到 0.01 米，继续减小 dt ，结果变化不大 (中图)。落地点距离随发射角的变化曲线出现明显的不对称 (右图)，最佳发射角偏离 $\pi/4$ 。

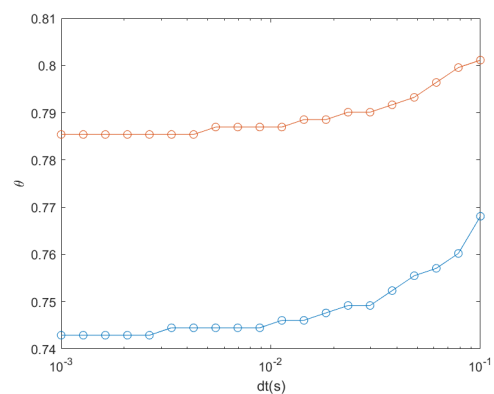


我们把之前找最佳发射角的程序，整合成函数，分析最后结果随 dt 和摩擦系数的关联。在下面的自定义函数中，输入 dt ，初始速度，摩擦系数，将返回对应的最佳发射角。

```
1 function y=findmaxfun(dt,v,b)
2 ts=linspace(0,pi/2,10);
3 for i=1:length(ts)
4     xl(i)=disfun2(ts(i),v,dt,b);
5 end
6 tt=linspace(0,pi/2,1000-1);
7 xx=spline(ts,xl,tt);
8 [q,qq]=max(xx);
9 y=tt(qq);
```

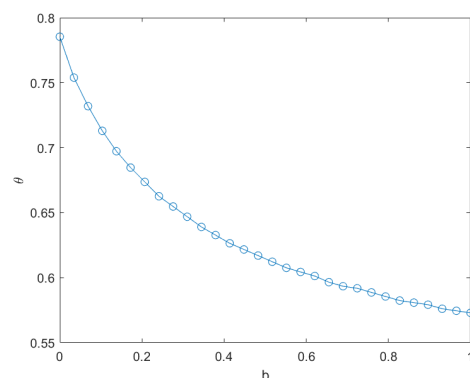
在下面的程序中，我们测试了不同 dt 对最佳发射角的影响。在考虑或者忽略空气阻力的情况下，最佳发射角都随 dt 的缩小而逐渐收敛，在 $dt=0.005$ 和 $dt=0.001$ 之间，就可以有较好的效果。当摩擦系数 $b=0.05$ ，最佳发射角比 $b=0$ 时要小，这是合理的，因为可以让沿水平方向的速度分量更大。

```
1 clear
2 dts=logspace(-1,-3,20);
3 for i=1:length(dts)
4     z(i)=findmaxfun(dts(i),10,0);
5     y(i)=findmaxfun(dts(i),10,0.05);
6 end
7 semilogx(dts,y,'o-')
8 hold on
9 semilogx(dts,z,'o-')
10 xlabel('dt(s)')
11 ylabel('\theta')
```



根据上面的测试，我们选择 $dt=0.001$ ，讨论了不同摩擦系数下的最佳发射角变化情况。从图看出，当摩擦系数从 $[0,1]$ 逐渐增大时，最佳发射角从 $\pi/4$ 逐渐减小到 0.57 ，可以判断是合理的结果。

```
1 clear
2 bs=linspace(0,1,30);
3 for i=1:length(bs)
4     y(i)=findmaxfun(0.001,10,bs(i));
5 end
6 plot(bs,y,'o-')
7 xlabel('b')
8 ylabel('\theta')
```



2.3 提高数值解法精度

在微分方程的初值问题中，解可以表示为积分形式的情况。比如，一阶线性常微分方程：

$$\frac{dy}{dt} + P(t)y = Q(t) \quad (2.22)$$

其解可以通过积分因子法表示为：

$$y(t) = \frac{1}{\mu(t)} \left(\int \mu(t)Q(t)dt + C \right) \quad (2.23)$$

其中，积分因子 $\mu(t)$ 定义为： $\mu(t) = \exp\left(\int P(t)dt\right)$ 。

在数值方法中，我们运用了有限划分，对积分计算和微分方程求解都带来误差。考虑到计算定积分和求解微分方程有一定的关联，下面通过相关案例讲解提高精度的思路和方法。一般情况，我们可以将划分变小来提高精度，但这样会明显增加计算量。因此，数值方法的发展需要在控制计算量的情况下，尽可能提高结果的精度。

2.3.1 Simpson 求积法

在很多实际问题中，我们是计算定积分，比如求面积、体积、微分方程等。如果我们知道被积函数的原函数，定积分计算非常容易。然而，多数时候被积函数没有简单的原函数，所以定积分需要借助数值方法。另外，数值方法能通过数目较少的采样积分点，实现对积分的合理估算，比如计算体积的万能公式（通过上、下底面、中间截面）等。

$$\int_a^b f(x)dx \cong \sum_{i=1}^{N-1} \frac{\Delta x}{2} [f(x_i) + f(x_{i+1})] \quad (2.24)$$

$$\cong \frac{\Delta x}{2} (f(a) + 2f(a + \Delta x) + 2f(a + 2\Delta x) + \dots + f(b)) \quad (2.25)$$

$$= \sum_{i=1}^N p_i * f_i = \frac{\Delta x}{2} \vec{p} \cdot \vec{f} \quad (2.26)$$

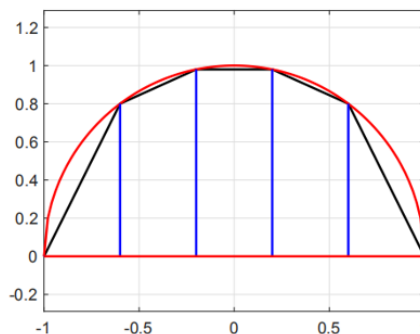
以一维情况为例，计算曲线和横轴围成的面积，可以通过定积分来实现。计算方案：将积分区间离散化（这里是把 $[a,b]$ 区间分成 $N-1$ 个等间距的部分， x 可能的取值有 N 个，对应的被积函数也有 N 个取值），计算曲线函数值将 $N-1$ 个梯形面积累加求和。如果将求和公式的同类项合并，可以得到积分公式， $\vec{p} = [0.5, 1, \dots, 1, 0.5]$, $\vec{f} = [f(a), \dots, f(b)]$ ，这里 $\vec{f}$ 是积分点的函数值， $\vec{p}$ 向量是权重。

下面的程序是梯形法的案例，第 3 行是被积函数，第 6、7 行都是输出积分结果，sum 是数组求和，第 7 行用的是行向量和列向量的矩阵乘法。

```

1 clear
2 x=linspace(-1,1,11);
3 y=(1-x.^2).^0.5;
4 p=ones(1,length(x));
5 p(1)=0.5;p(end)=0.5;
6 sum(y.*p)*(x(2)-x(1))
7 y*p'*(x(2)-x(1))

```



和公式 2.27 不同，下面给出在 Simpson 方法的计算方式：

$$\int_a^b f(x)dx \cong \frac{\Delta x}{3}(f(a) + 4f(a + \Delta x) + 2f(a + 2\Delta x) + 4f(a + 3\Delta x) + \dots + f(b)) \quad (2.27)$$

为了理解 Simpson 方法，选择相邻的 3 个积分点 $(a, f(a)), (\frac{a+b}{2}, f(\frac{a+b}{2})), (b, f(b))$ ，通过二次插值得到对应的多项式：

$$f(x) = f(a) \frac{(x - \frac{a+b}{2})(x - b)}{(a - \frac{a+b}{2})(a - b)} + f\left(\frac{a+b}{2}\right) \frac{(x - a)(x - b)}{(\frac{a+b}{2} - a)(\frac{a+b}{2} - b)} + f(b) \frac{(x - \frac{a+b}{2})(x - a)}{(b - \frac{a+b}{2})(b - a)} \quad (2.28)$$

并对该多项式进行积分：

$$\int_a^b f(x)dx = f(a)I(a) + f\left(\frac{a+b}{2}\right)I\left(\frac{a+b}{2}\right) + f(b)I(b) \quad (2.29)$$

其中，分别对三个二次函数进行积分：

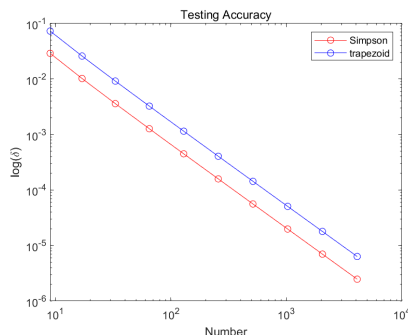
$$\begin{cases} I(a) = \int_a^b \frac{(x - \frac{a+b}{2})(x - b)}{(a - \frac{a+b}{2})(a - b)} dx = \frac{b-a}{6} \\ I\left(\frac{a+b}{2}\right) = \int_a^b \frac{(x-a)(x-b)}{(\frac{a+b}{2} - a)(\frac{a+b}{2} - b)} dx = \frac{2(b-a)}{3} \\ I(b) = \int_a^b \frac{(x - \frac{a+b}{2})(x - a)}{(b - \frac{a+b}{2})(b - a)} dx = \frac{b-a}{6} \end{cases} \quad (2.30)$$

可以看到，积分权重变成 1: 4: 1。对于一般的体积公式，我们可以用上、下底面 (b_1, b_3) 和中截面面积 (b_2) 、高 (h) 来估算： $V = \frac{h}{6}(b_1 + 4b_2 + b_3)$ 。计算体积的万能公式是 Simpson 方法的特例。当推广到 $2N+1$ 个积分点时，我们可以分成 N 组，每组有 3 个点，而且相邻两组有共用点，可以得到 1:4:1, 1:4:1 的积分值，把共用的积分点合并，权重就是 1:4:2:4:2:1。Simpson 方法权重为：首尾 2 项的系数是 $1/3$ ；其它的偶数项的系数是 $4/3$ ；奇数项的系数是 $2/3$ 。下面左边程序是比较梯形法和 Simpson 方法计算半圆面积误差随积分点数目的变化。从右图看出，在相同数目的积分点时，Simpson 方法误差更小，比梯形法更高效。

```

1 clear
2 Ns=2.^(3:12)+1;
3 for i=1:length(Ns)
4 s(i)=Simpsonfun(Ns(i));
5 t(i)=circlefun(Ns(i));
6 end
7 loglog(Ns,abs(s-pi/2),'ro-')
8 hold on
9 loglog(Ns,abs(t-pi/2),'bo-')

```



2.3.2 Runge-Kutta 方法

对于运动微分方程, 根据受力分析, 可以得到: $\frac{d^2s}{dt^2} = a(s, v, t)$, 这里的加速度 a 可以是位移 s 、速度 v 、时间 t 的函数。常用的差分形式是先更新速度 $v_{i+1} = v_i + a_i(s_i, v_i, t_i)\Delta t$, 然后根据速度更新 $s_{i+1} = s_i + v_i\Delta t$ 。注意这里用 v_i 还是 v_{i+1} 对结果是有影响的, 此外, 我们还可以用 v_i 和 v_{i+1} 的平均值来减少误差, $s_{i+1} = s_i + \frac{(v_i + v_{i+1})\Delta t}{2}$ 。类似的, 根据加速度更新速度时, 我们用平均值来计算: $v_{i+1} = v_i + \frac{(a_i + a_{i+1})\Delta t}{2}$ 。下面以有空气阻力的落体运动为例, 看看不同的更新方案对结果的影响。假定阻力大小是速度的一次函数, 微分方程和解如下:

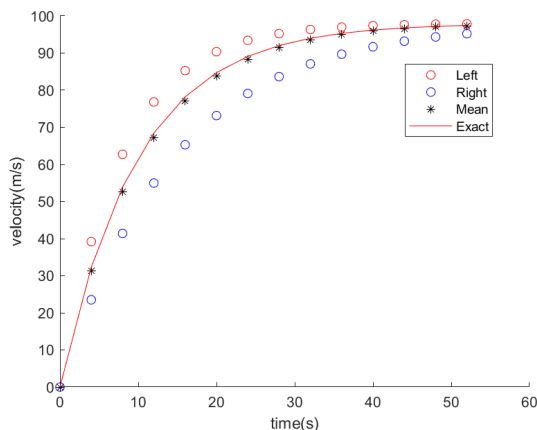
$$\frac{dv}{dt} = g - bv \rightarrow v = \frac{g}{b}(1 - \exp(-bt)) \quad (2.31)$$

第 8 行程序对应的是取加速度平均值的更新方式 (Mean), 可以修改成不同的更新方法, 其中 Left 对应 $v(j+1) = v(j) + a(j) * dt$, Right 对应 $v(j+1) = v(j) + a(j+1) * dt$ 。

```

1 clear
2 g=9.8; dt=4; b2m=0.1;
3 t(1)=0; v(1)=0; s(1)=0; a(1)=g;
4 for j=1:round(50/dt)
5     a(j)=g-b2m*v(j);
6     v(j+1)=v(j)+a(j)*dt;
7     a(j+1)=g-b2m*v(j+1);
8     v(j+1)=v(j)+(a(j+1)+a(j))*dt/2;
9     t(j+1)=t(j)+dt;
10 end
11 plot(t,v,'k*')
12 hold on
13 plot(t,g*(1-exp(-b2m*t))/b2m,'r')

```



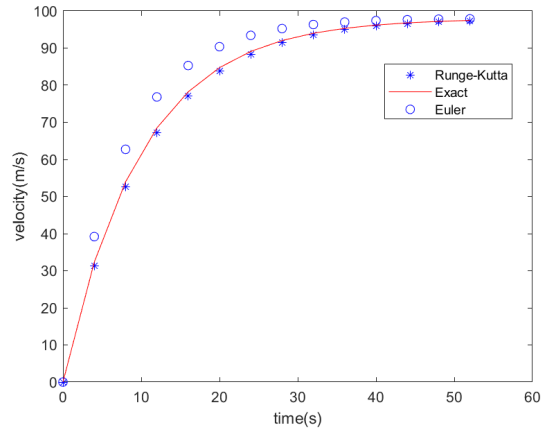
一般来说, 用差分方式 (Euler 法) 解微分方程时, 如果对自变量进行细分, 比如 dt 从 0.5 到 0.05, 解的精度将明显提高, 但同时计算耗时明显增加。有效的方法要求在不改变 dt 的情况下, 提高解的精度。由于自变量的离散化是有限大小, 误差来自于高阶项的舍去。如果能考虑高阶项的贡献, 可以有效提高精度。而高阶项对应微分方程的高阶导数, 直接计算是比较麻烦的。

为了提高计算精度，我们这里介绍 Runge-Kutta 方法，二阶形式如下：

$$x(t + \Delta t) = x(t) + f(x', t')\Delta t, \quad x' = x(t) + \frac{1}{2}f(x(t), t)\Delta t, \quad t' = t + \frac{1}{2}\Delta t \quad (2.32)$$

和 Euler 方法类似，不过要提前计算 $f(x', t')$ ，而 x' 的计算公式是对应 Euler 方法中 Δt 变成原来的一半。如下图所示，相同的 dt 情况下，Runge-Kutta 方法结果比欧拉法更接近严格解，证实了算法在精度提高的有效性。

```
1 clear
2 g=9.8; dt=4; b2m=0.1;
3 t(1)=0; v(1)=0; s(1)=0; a(1)=g;
4 for j=1:round(50/dt)
5     RK=v(j)+(g-b2m*v(j))*dt/2;
6     v(j+1)=v(j)+(g-b2m*RK)*dt;
7     t(j+1)=t(j)+dt;
8 end
9 plot(t,v,'*b')
10 hold on
11 plot(t,g*(1-exp(-b2m*t))/b2m,'r')
12 xlabel('time(s)')
13 ylabel('velocity(m/s)')
```



下面给出简单的证明。我们知道，方程的解可以展开为：

$$x(t+\Delta t) = x(t) + \frac{dx}{dt}\Delta t + \frac{1}{2}\frac{d^2x}{dt^2}(\Delta t)^2 + \dots \rightarrow x(t_{i+1}) \approx x(t_i) + f_i\Delta t + \frac{1}{2}\left(\left.\frac{\partial f}{\partial t}\right|_i + f_i\left.\frac{\partial f}{\partial x}\right|_i\right)(\Delta t)^2 \quad (2.33)$$

其中，一阶导数 $\frac{dx}{dt} = f(x, t)$ 对应的是微分方程，但通常不能通过计算高阶导数来提高精度。根据多元函数的全微分 $Z(X + \Delta X, Y + \Delta Y) = Z(X, Y) + \frac{\partial Z}{\partial X}\Delta X + \frac{\partial Z}{\partial Y}\Delta Y$ ，我们发现 RK 公式可以展开为：

$$f(x', t') = f\left(x + \frac{1}{2}f(x, t)\Delta t, t + \frac{1}{2}\Delta t\right) \quad (2.34)$$

$$= f(x, t) + \frac{\partial f}{\partial x}\left(\frac{1}{2}f(x, t)\Delta t\right) + \frac{\partial f}{\partial t}\left(\frac{1}{2}\Delta t\right) \quad (2.35)$$

$$= f(x, t) + \frac{1}{2}\left(\frac{\partial f}{\partial t} + f\frac{\partial f}{\partial x}\right)\Delta t \quad (2.36)$$

可见，RK 方法表达式的展开项包含了微分方程解的二阶展开式，所以精度提高。值得注意的是，RK 方法只利用了微分方程中的一阶导数，避免了计算高阶导数，而展开式能出现高阶项，这是巧妙之处。

对于更一般的情况, Runge-Kutta 方法根据微分方程构造一阶导数的线性组合, 通过待定系数法得到不同精度的表达式。假定体系的严格解可以表示为:

$$x(t + \Delta t) = x(t) + \alpha_1 c_1 + \alpha_2 c_2 + \dots + \alpha_m c_m \quad (2.37)$$

其中 $c_1 = \Delta t f(x(t), t)$, $c_2 = \Delta t f(x(t) + \nu_{21} c_1, t + \nu_{21} \Delta t)$, 从 α_1 到 α_m 和 ν_{21} 都是待定系数, 就相当于仅用了一阶导数 $f(x, t)$ 来构造方程的解。为了在展开式中出现 Δt 的高次方, 在 c_2 的 x, t 位置中出现了 c_1 和 Δt 。简单起见, 我们以二阶形式为例, 看看如何确定系数。此时, 方程的解为:

$$x(t + \Delta t) = x(t) + \alpha_1 c_1 + \alpha_2 c_2, \quad c_1 = \Delta t f(x(t), t), \quad c_2 = \Delta t f(x + \nu_{21} c_1, t + \nu_{21} \Delta t) \quad (2.38)$$

可以根据展开式精度确定系数 α_1, α_2 和 ν_{21} , 这里比较关键的是 c_2 的展开, δt 是小量, 全微分展开为: $c_2 = f \Delta t + \nu_{21} \Delta t^2 \left(\frac{\partial f}{\partial t} + f \frac{\partial f}{\partial x} \right)$ 。根据公式 2.33, 我们可以得到系数满足的方程:

$$\alpha_1 + \alpha_2 = 1, \quad \alpha_2 \nu_{21} = \frac{1}{2} \quad (2.39)$$

选择 $\alpha_1 = 0, \alpha_2 = 1, \nu_{21} = \frac{1}{2}$, 这样就得到了我们之前给出的 RK 方法对应的公式 2.32。我们还可以选择 $\alpha_1 = \alpha_2 = \frac{1}{2}, \nu_{21} = 1$, 对应的就是之前用的平均值方法。